
UniMixer：面向推荐系统扩展定律的统一架构

Mingming Ha Guanchen Wang Linxun Chen Xuan Rao Yuexin Shi Tianbao Ma
Zhaojie Liu Yunqian Fan Zilong Lu Yanan Niu Han Li Kun Gai

快手科技，北京，中国

{hamingming, wangguanchen, chenxi36, raoxuan, shiyuexin, matianbao,
zhaotianxing, fanyunqian03, luzilong, niuyanan, lihan08}@kuaishou.com, gaikun@qq.com

Abstract

近年来，推荐模型的扩展定律日益受到关注，该定律刻画了推荐模型性能与参数量/FLOPs 之间的关系。目前，实现推荐模型扩展主要有三类主流架构，即基于注意力、基于 TokenMixer 和基于因子分解机的方法，它们在设计理念与架构结构上均存在根本差异。本文提出一种面向推荐系统的统一扩展架构 **UniMixer**，旨在提升扩展效率，并建立统一主流扩展模块的理论框架。通过将基于规则的 TokenMixer 转化为等价的参数化结构，我们构建了一个广义参数化特征混合模块，使 token 混合模式能够在模型训练过程中得到优化和学习。同时，广义参数化 token 混合解除了 TokenMixer 中头数必须等于 token 数的约束。此外，我们建立了推荐系统扩展模块的统一设计框架，打通了基于注意力、基于 TokenMixer 与基于因子分解机的方法之间的联系。为进一步提升扩展投入产出比，我们设计了轻量级 UniMixing 模块 **UniMixing-Lite**，在显著提升模型性能的同时，进一步压缩模型参数量与计算成本。扩展曲线如下图所示。大量离线与在线实验验证了 **UniMixer** 卓越的扩展能力。

1 引言

大语言模型 (LLM) 揭示了一个令人瞩目的现象：随着模型规模、数据量和计算资源增加，模型性能稳步提升，即扩展定律。LLM 中观察到的显著性能扩展现象，促使推荐系统社区探索适用于推荐任务的扩展框架。近年来，研究人员尝试设计扩展模块并将其堆叠为多层，以提升排序模型的复杂度，从而建立模型性能与模型规模或计算成本（例如参数量和 FLOPs）之间的扩展定律。

推荐系统基于大量多特征域的用户和物品特征预测用户行为，向用户呈现最相关的内容，以提升用户对推荐结果的正向互动。这些多特征域特征通常包含类别特征和稠密特

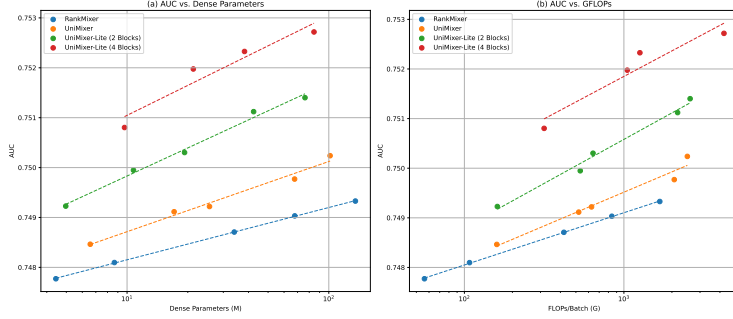


图 1: 本文所提出的 UniMixer/UniMixer-Lite 与 RankMixer 架构中, AUC 与参数量之间的扩展定律。横轴采用对数尺度。

征, 往往具有更动态的嵌入表示, 并从多个视角捕获信息。在自然语言处理 (NLP) 领域, 所有 token 共享统一的嵌入空间; 与之不同, 推荐任务中的特征空间在本质上具有异构性。因此, 学习异构特征交互构成了推荐任务与 NLP 领域之间的根本差异。鉴于 Transformer 在 LLM 中取得了巨大成功, 一个自然的思路是修改 Transformer 模块以适配推荐任务, 因为通常无法直接将 Transformer 模块用作推荐系统扩展定律的基础模块。为解决异构特征交互问题, 当前推荐模型的主流扩展架构大体可分为三类: 基于注意力、基于 TokenMixer 和基于因子分解机的方法。基于注意力的方法 (例如 HiFormer [1]、FAT [2] 和 HHFT [3] 等) 为每个输入 token 构建 token 特定的 query、key 和 value 投影。与基于注意力的方法不同, 基于 TokenMixer 的方法 (例如 RankMixer [4]、TokenMixer-Large [5] 等) 采用基于规则的 token 混合操作来实现异构特征交互, 从而避免计算两个异构语义空间之间的内积相似度。另一方面, 基于因子分解机的方法 (例如 Wukong [6]、Kunlun [7] 等) 通过引入因子分解机 (FM) 模块, 计算每一层内输入嵌入之间的交互, 从而对特征交互进行建模。这些框架建立在完全不同的扩展模块之上, 却都展现出扩展模型性能的能力。这引出了一个根本问题: 我们能否为推荐系统构建一个融合现有主流扩展组件优势的统一扩展模块? 为打通这些扩展模块之间的联系, 我们首先为基于规则的 TokenMixer 操作找到一种参数化表述。通过进一步优化计算流水线, 我们推导出计算成本更低的 UniMixing 模块。在这一设计和结果的基础上, 我们提出一个统一理论框架, 将推荐系统中的主流扩展模块统一起来。此外, 我们还设计了一个轻量级 UniMixer 模块, 该模块利用现有主流扩展模块的优势, 并实现最佳的参数效率与计算效率。我们希望这一统一架构能够帮助推荐系统社区迎来属于自己的 “attention moment”。我们的主要贡献可概括如下:

- 1) 我们通过对基于规则的 TokenMixer 进行等价参数化, 揭示了 TokenMixer 的特征交互模式。
- 2) 我们提出了名为 UniMixer 的统一扩展框架, 打通了基于注意力、基于 TokenMixer 和基于 FM 的方法之间的差异与联系。通过优化计算流水线, UniMixer 显著降低了训练与推理过程中的计算复杂度和 GPU 显存消耗。

3) 为进一步减少模型参数量和计算成本,我们设计了名为 UniMixing-Lite 的轻量级 UniMixing 模块,该模块可同时发挥基于注意力与基于 TokenMixer 架构的优势,从而提升扩展效率。

4) 大量离线与在线实验验证了 UniMixer 卓越的扩展能力。

2 相关工作

目前,为大规模推荐系统建立扩展定律主要有三种扩展建模范式:基于注意力、基于 TokenMixer 和基于 FM 的方法。

基于注意力的框架。 近期推荐系统研究已将 Transformer 用于 CTR 预测。这一范式的核心挑战在于,如何弥合 token 序列的异质本质与语言建模所假设的序列组合性之间的差距。为此, [1] 提出了异质注意力层以处理异质特征交互,并设计 HiFormer,通过将异质 token 展平为单个向量表示来显式建模高阶交互。此外, Field-Aware Transformers (FAT) 通过因子化上下文对齐与跨域调制,将特征域感知的交互先验注入注意力机制 [2],进一步建立了 CTR 预测的经验扩展定律。HHFT 通过交替堆叠异质 Transformer 模块(用于保留领域特定语义)与 HiFormer 模块(用于学习高阶交互),进一步验证了这些扩展性质 [3]。此外,在动态用户行为建模中, HSTU-V1/V2 [8, 9]、MARM [10]、OneTrans [11]、Climber [12]、Hyformer [13] 和 LLaTTE [14] 等方法利用注意力机制捕获长程时间依赖。这些方法凸显了统一特征交互与序列行为建模以获得更稳健扩展定律的潜力。

基于 TokenMixer 的框架。 尽管注意力机制能够实现富有表现力的特征交互,但注意力分数计算的二次复杂度会带来高昂的计算成本。受 MLP-Mixer [15] 在计算机视觉领域成功的启发,工业推荐系统开始转向 token 混合架构,并涌现出 RankMixer [4]、Lemur [16] 和 TokenMixer-Large [5] 等先进模型。例如, RankMixer 以静态、非参数化的 token 混合操作取代动态注意力,在保持 FLOPs 严格可比的同时,实现了具有竞争力的 CTR 预测性能 [4]。在此基础上, TokenMixer-Large 通过引入辅助残差连接与定制损失函数,将该架构扩展至 13B 配置 [5],并在不同模型维度上展现出显著的扩展定律。然而,一个关键缺口依然存在:当前 token 混合算子的设计高度依赖经验规则,缺乏与传统的基于 FM 或基于注意力方法之间的严格理论桥梁。

基于 FM 的框架。 开创性的 FM 方法采用低阶成对建模来处理推荐系统中的特征交互 [17],随后 Field-aware FMs 对其进行了推广,以捕获特征域特定且上下文敏感的交互 [18]。尽管这些模型具有高可解释性和高效率,但其低阶交互能力从根本上限制了模型容量。为解决这一局限, DeepFM [19]、AutoInt [20] 和 DCN 系列 [21, 22] 等多种基于神经网络的扩展方法集成了 MLP 或 Transformer 注意力,以捕获高阶交互。近来, Wukong [6] 通过堆叠 FM 风格的交互模块与线性压缩,展现出适当的扩展性质。然而,基于 FM 的方法对显式低阶交互的依赖,仍然限制了模型在参数量和 FLOPs 扩大时的性能提升,这与 LLM 中观察到的可预测扩展定律形成对比 [23, 24]。

3 预备知识

考虑一类判别式推荐任务，例如评分、点击率（CTR）和点击后转化率（CVR）预测等，这些任务通常被表述为监督学习问题。数据集定义为 $\mathcal{D} = \{(\mathbf{X}_1, y_1), \dots, (\mathbf{X}_i, y_i), \dots, (\mathbf{X}_N, y_N)\}$ ，其中 $\mathbf{X}_i = [\mathbf{x}_i^{(1)}, \mathbf{x}_i^{(2)}, \dots, \mathbf{x}_i^{(F)}]$ 包含 F 个特征域； $y_i \in \{0, 1\}$ （对应二分类问题）或 $y_i \in \mathbb{R}$ （对应回归问题）是第 i 个样本的标签； N 是数据点数量。通常，输入特征 $\mathbf{X} = \{\mathbf{X}^C, \mathbf{X}^D\}$ 被划分为类别特征 $\mathbf{X}^C$ 和稠密特征 $\mathbf{X}^D$ 。 $|\mathbf{X}^C|$ 和 $|\mathbf{X}^D|$ 分别表示类别特征与稠密特征的数量。对于 CTR 和 CVR 预测任务，核心目标是建立模型来预测点击或转化概率 $\Pr(y_i = 1 | \mathbf{X}_i)$ 。在推荐系统中，学习到的嵌入表示具有更强的动态性 [1]。与语言模型的输入 token 不同，特征空间在本质上具有异质性 [4]。因此，将大语言模型所使用的 Transformer 架构直接迁移到推荐建模中并不合适。迄今为止，推荐领域的扩展定律主要通过三类基础模块及其变体建立。

异质注意力层。 基于异质注意力的架构 [1, 2, 3] 通常采用特征域特定的 query、key 和 value 投影来实现异质特征交互。给定输入隐藏状态 $X = [\mathbf{x}_1; \dots; \mathbf{x}_T] \in \mathbb{R}^{T \times D}$ ，异质注意力层可表示为

$$Q_h = \begin{bmatrix} \mathbf{x}_1 W_Q^{1h} \\ \vdots \\ \mathbf{x}_T W_Q^{Th} \end{bmatrix} \in \mathbb{R}^{T \times d}, K_h = \begin{bmatrix} \mathbf{x}_1 W_K^{1h} \\ \vdots \\ \mathbf{x}_T W_K^{Th} \end{bmatrix} \in \mathbb{R}^{T \times d}, V_h = \begin{bmatrix} \mathbf{x}_1 W_V^{1h} \\ \vdots \\ \mathbf{x}_T W_V^{Th} \end{bmatrix} \in \mathbb{R}^{T \times d}, \quad (1)$$

其中， $W_Q^{ih}, W_K^{ih}, W_V^{ih} \in \mathbb{R}^{D \times d}$ 分别为 query、key 和 value 投影的 token 特定权重。多头异构注意力层的输出计算如下

$$O_h = \text{softmax}\left(\frac{Q_h K_h^T}{\sqrt{d}}\right) V_h \in \mathbb{R}^{T \times d}. \quad (2)$$

随后，将多头异构注意力的输出拼接起来，并通过线性投影使输出维度与输入 X 对齐。

TokenMixer。 基于 TokenMixer 的框架 [4, 5, 25] 采用无参数且基于规则的混合操作来执行特征交互。对于给定输入 $X = [\mathbf{x}_1; \dots; \mathbf{x}_T]$ ，TokenMixer 首先将每个输入 token $\mathbf{x}_t$ 均匀划分为 H 个头。

$$[\mathbf{x}_t^{(1)} | \mathbf{x}_t^{(2)} | \dots | \mathbf{x}_t^{(H)}] = \text{SplitHead}(\mathbf{x}_t). \quad (3)$$

随后，第 h 个 token $\mathbf{s}^h$ 可表示为

$$\mathbf{s}^h = \text{concat}(\mathbf{x}_1^{(h)}, \mathbf{x}_2^{(h)}, \dots, \mathbf{x}_T^{(h)}) \in \mathbb{R}^{\frac{TD}{H}} \quad (4)$$

TokenMixer 的输出可表示为

$$S = \begin{bmatrix} \mathbf{s}_1 \\ \vdots \\ \mathbf{s}_H \end{bmatrix} \in \mathbb{R}^{H \times \frac{TD}{H}}, \quad (5)$$

其中， H 必须与 T 相同。因此，输入 X 与输出 S 的维度一致。

Wukong. 基于 Wukong 的模型 [6, 7] 将因子分解机模块 (FMB) 的输出与线性投影层的输出拼接起来, 以扩大交互组件的维度。

$$\begin{aligned} \text{FMB}(X) &= \text{reshape}(\text{MLP}(\text{LN}(\text{flatten}(\text{FM}(X))))), \text{FM}(X) = XX^T Y \\ \text{LCB}(X) &= WX \end{aligned} \quad (6)$$

其中 $W \in \mathbb{R}^{n \times T}$ 和 $Y \in \mathbb{R}^{T \times r}$ 是可学习的投影矩阵。 Y 用于减少存储交互矩阵 XX^T 所需的内存。

在本工作中, 我们致力于为推荐系统建立统一的结构基础, 集成当前各种规模化块的优势, 以进一步提高规模化的 ROI。

4 UniMixer

4.1 概述

我们建立了一个用于扩展推荐系统的统一模块, 即 UniMixer 块, 它在统一的理论框架下整合了推荐领域的主流规模化模块, 如基于注意力的模块、基于 TokenMixer 的模块和基于 Wukong 的方法。如图 2 所示, 整体架构包括特征 token 化、 M 个带有 Siamese norm 和 Sparse-Pertoken MoE 的 UniMixer 块。通过对基于规则的 TokenMixer 进行参数化, 我们在基于注意力、基于 TokenMixer 和基于 Wokong 的方法之间建立联系, 使所提出的 UniMixer 能够同时具备这些方法的优势。此外, 我们还开发了轻量级 UniMixing 模块, 在显著提升模型性能的同时, 进一步压缩模型参数和计算成本。

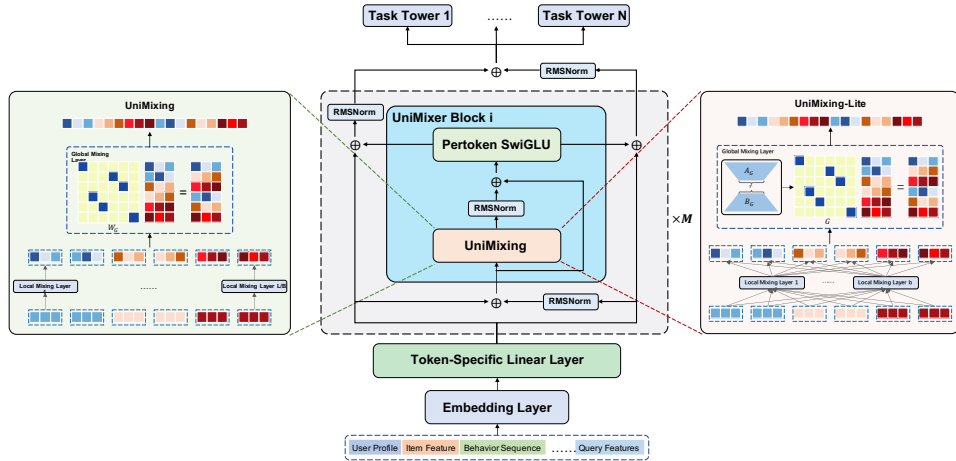


图 2: 用于推荐系统规模定律的 UniMixer 架构。

4.2 特征 Token 化

根据输入特征字段的语义类别, 首先将输入特征 $\mathbf{X}$ 划分为 N 个互不相交的特征域

$$\mathbf{X} = \left[\underbrace{\mathbf{x}_U^{(1)}, \dots, \mathbf{x}_U^{(n_U)}}_{\text{User Profile}}, \underbrace{\mathbf{x}_I^{(1)}, \dots, \mathbf{x}_I^{(n_I)}}_{\text{Item Features}}, \underbrace{\mathbf{x}_B^{(1)}, \dots, \mathbf{x}_B^{(n_B)}}_{\text{Behavior Sequence}}, \underbrace{\mathbf{x}_Q^{(1)}, \dots, \mathbf{x}_Q^{(n_Q)}}_{\text{Query Features}}, \dots \right]. \quad (7)$$

每个特征域都通过 embedding 层转换为不同的嵌入向量

$$\mathbf{e}_n = \text{Embedding}(\mathbf{X}_{\text{domain}}) \in \mathbb{R}^{d_{\text{domain}}}, \quad (8)$$

其中 $\mathbf{X}_{\text{domain}}$ 表示一个特征域内的所有特征, d_{domain} 是该特征域对应的嵌入维度。将所得特征域嵌入拼接成一个嵌入向量 $\mathbf{E} = [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_N]$ 。与 [4] 类似, 将嵌入向量 $\mathbf{E}$ 均匀划分为适当数量的块。然后, 使用以下 token 特定线性层将每个块投影为 token 嵌入

$$\mathbf{x}_i = W_i^{\text{proj}} \mathbf{E}_{di:di+d} + \mathbf{b}_i^{\text{proj}} \in \mathbb{R}^D, \quad (9)$$

其中 $W_i^{\text{proj}} \in \mathbb{R}^{D \times d}$, $\mathbf{b}_i^{\text{proj}} \in \mathbb{R}^D$ 。随后, 通过按列堆叠 $\mathbf{x}_i$ 可得到输入隐状态 $X \in \mathbb{R}^{T \times D}$ 。

4.3 UniMixer 块

异构特征交互。 如第 3 节所述, 异构注意力通过使用 token 特定的查询、键和值权重, 解决两个异构语义空间之间的特征交互问题。然而, 通过计算内积相似度得到的注意力模式通常带有对角占优的先验。在训练早期, 对于随机初始化的权重 W_Q^h 和 W_K^h , 注意力权重的量级在很大程度上受输入 token 值 X 主导, 这很容易使注意力权重集中在少数 token 上, 如图 3(a) 所示。

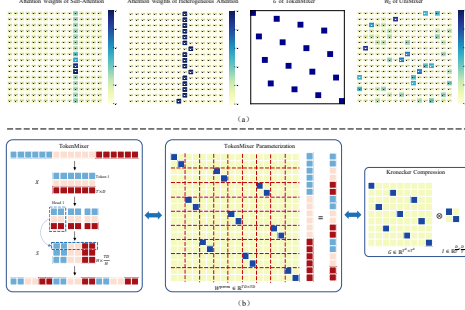


图 3: (a) 不同方法的全局混合权重。(b) 基于规则的 TokenMixer 的等价参数化。

如图 3(a) 所示, 可以观察到, 异构注意力的注意力权重尖锐且稀疏, 这会给梯度反向传播带来风险, 使 query 和 key 权重难以训练, 甚至可能导致训练停滞, 如图 3(a) 所示 (异构注意力的注意力权重中第 10 行和第 15 行)。同时, 在大规模异构特征输入下, 此类注意力模式可能导致特征交互趋于均匀, 即注意力分数变得很小且缺乏区分度, 进而可能产生噪声信号, 掩盖关键的特征交互模式。

另一方面, 无参数且基于规则的 TokenMixer 操作缺乏可学习性和场景适应性, 可能导致异构特征交互不足或错误。此外, 要求 $T = H$ 进一步限制了异构特征交互模式的选择。通过深入分析 TokenMixer 操作, 我们得到了一些有趣的发现, 使 TokenMixer 操作的参数化成为可能。如图 3(b) 所示, 我们观察到, TokenMixer 操作可视为置换矩阵 W^{perm} 与展平后的输入嵌入 $\text{flatten}(X) \in \mathbb{R}^{TD}$ 的乘积, 其形式为

$$\text{TokenMixer}(X) = \text{reshape}(W^{\text{perm}} \text{flatten}(X)), \quad (10)$$

其中 $W^{\text{perm}} \in \mathbb{R}^{TD \times TD}$ 是一个大型置换矩阵。附录 A 中给出了一个具体的数值示例。一个自然的想法是通过将置换矩阵 W^{perm} 进行参数化, 使基于规则的 TokenMixer 可学习且可优化。然而, 其计算复杂度 $O(T^2 D^2)$ 和参数量 $O(T^2 D^2)$ 都令人无法接受。通

过观察，我们对 TokenMixer 的置换矩阵 W^{perm} 得出了一些有趣的发现，并将其概括为下框中的启发性质。

置换矩阵 W^{perm} 的性质。

- **可压缩性。** 置换矩阵 W^{perm} 可等价分解为两个较小矩阵的 Kronecker 积，即 $W^{\text{perm}} = G \otimes I$ ，其中 $I \in \mathbb{R}^{\frac{D}{T} \times \frac{D}{T}}$ 是单位矩阵， $G \in \mathbb{R}^{T^2 \times T^2}$ ， $\otimes$ 是 Kronecker 积运算。
- **双随机性。** 在置换矩阵 W^{perm} 中，每行和每列的元素之和均为 1，即 $\sum_{p=1}^{TD} w_{pq}^{\text{perm}} = 1, \sum_{q=1}^{TD} w_{pq}^{\text{perm}} = 1$ 。
- **稀疏性。** W^{perm} 的每一行/列恰好包含一个非零元素。
- **对称性。** 若 $T = H$ ，则置换矩阵 W^{perm} 是对称矩阵，即 $W^{\text{perm}} = W^{\text{perm}^T}$ 。若 $T \neq H$ ，则置换矩阵 W^{perm} 是非对称矩阵。

根据 TokenMixer 置换矩阵的性质，通过对矩阵 G 和 I 进行参数化，可显著减少 token 混合所需的参数量，即 $O(T^4 + (\frac{D}{T})^2)$ ，其中 T 通常远小于 D 。此外，TokenMixer 的参数化仍面临三个挑战：(1) 直接使用参数化的 G 和 I 重建 W^{perm} ，在模型训练和推理过程中仍会产生大小为 $[TD, TD]$ 的中间变量，对 GPU 内存提出了非常高的要求；(2) 如何确保学得参数满足双随机性、稀疏性和对称性；(3) 如何设计一个整合现有规模化模块优势的统一推荐规模化模块，并为推荐系统建立更优的规模化效率。

统一 Token 混合模块。 受图 3 启发，在统一 token 混合模块中，不再使用 T 和 D ；取而代之的是，我们在置换矩阵中定义块和块大小。块大小记为 B 。块数量为 $(L//B)^2$ ，其中 L 是输入嵌入维度，并且能被块大小 B 整除。将 G 的参数化权重记为 W_G 。考虑到置换矩阵的稀疏性，并为了实现充分的异构特征交互，我们为每一行分配不同的参数化权重 W_B^i 。通过该操作，每个块都具有不同的特征交互模式。然后，通过学习参数矩阵 W_G 和 W_B^i ，可得到具有更丰富交互模式的置换矩阵 W^{perm} ，其表达式为

$$\text{UniMixing}(X) = \text{reshape}\left(\left(W_G \otimes \{W_B^i\}_{i=1}^{L//B}\right) \text{flatten}(X), 1, L\right), \quad (11)$$

其中， $\otimes$ 为广义 Kronecker 积。

接下来，对 (11) 的计算流程进行优化，以显著降低计算成本与 GPU 显存需求。首先将嵌入向量 $\text{flatten}(X)$ 均匀划分为 $L//B$ 个向量，每个向量的大小为 B ，可表示为

$$\left[\mathbf{x}_1 \mid \mathbf{x}_2 \mid \dots \mid \mathbf{x}_{\frac{L}{B}}\right] = \text{Split}\left(\text{flatten}(X), \frac{L}{B}\right). \quad (12)$$

然后，将块权重 W_B^i 分别与对应的分块向量 $\mathbf{x}^{(i)}$ 相乘，得到以下局部特征交互向量

$$\text{reshape}\left(H, \frac{L}{B}, B\right) = \text{reshape}\left(\left[\mathbf{x}_1 W_B^1 \mid \mathbf{x}_2 W_B^2 \mid \dots \mid \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}}\right], \frac{L}{B}, B\right) = \begin{bmatrix} \mathbf{x}_1 W_B^1 \\ \vdots \\ \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}} \end{bmatrix}. \quad (13)$$

因此，UniMixing 模块的输出可表示为

$$\text{UniMixing}(X) = \text{reshape}\left(W_G \text{reshape}\left(H, \frac{L}{B}, B\right), 1, L\right). \quad (14)$$

通过该操作，与直接使用重建矩阵 W^{perm} 相比，优化后的计算流水线将计算成本从 $O(L^2)$ 降低到 $O(L^2/B + LB)$ ，并避免在计算过程中创建大型中间变量。式 (11) 的计算流水线优化证明见附录 B。根据 (13) 和 (14)， W_B^i 控制块内交互模式，而 W_G 控制块间交互模式。对于维度为 L 的嵌入输入，不再要求 $T = H$ 。与 TokenMixer 操作相比，UniMixing 模块具有更多样的局部和全局特征混合模式及交互尺度，同时仍保留了可学习和可优化的优势。为了使学得置换矩阵满足双随机性，使用 Sinkhorn-Knopp 迭代：先通过指数算子使 W_G 和 W_B^i 的所有元素均为正，然后进行迭代归一化，交替缩放各行和各列，使其元素之和为 1。此外，我们引入温度系数来控制参数矩阵的稀疏性。最后，我们使用 $(W_G + W_G^T)/2$ 和 $(W_B^i + W_B^{iT})/2$ 实现参数矩阵的对称性约束。最终的受约束权重可通过下式得到

$$\begin{aligned} \tilde{W}_G &= \frac{W_G + W_G^T}{2}, \quad \tilde{W}_B^i = \frac{W_B^i + W_B^{iT}}{2}, \\ \bar{W}_G &= \text{Sinkhorn-Knopp}\left(\frac{\tilde{W}_G}{\tau}\right), \quad \bar{W}_B^i = \text{Sinkhorn-Knopp}\left(\frac{\tilde{W}_B^i}{\tau}\right), \end{aligned} \quad (15)$$

其中， τ 为温度系数。

随后，使用残差连接与归一化模块处理 UniMixing 模块的输出

$$O = \text{RMSNorm}(X + \text{UniMixing}(X)) \quad (16)$$

异质特征交互的统一视角。 观察 (1) 中的 V_h 与 (13) 中的 $\text{reshape}(H, L/B, B)$ ，我们发现，若将块数 L/B 设为 T ，且 W_V^{ih} 与 W_B^i 具有相同维度，则有 $\text{reshape}(H, L/B, B) = V_h$ 。这意味着，在 $W_V^i = W_B^i$ 条件下，UniMixer 的局部交互投影等价于异质注意力层的 value 投影。另一方面， W_G 的维度与作用均与注意力权重相同，但 W_G 还需满足双随机性、稀疏性和对称性。Wukong 的特征交互基于 FM 组件。根据 (6)， $\text{FMB}(X)$ 的表达式可重写为 $\text{FMB}(X) = \text{reshape}(\text{MLP}(\text{LN}(\text{flatten}(XI(XI)^T Y))))$ ，其中 I 是具有适当维度的单位矩阵。下面聚焦核心特征交互模块 $XI(XI)^T Y$ 。在注意力模块中，当 $W_Q = I$ 、 $W_K = I$ ，且 value 矩阵不依赖隐藏状态输入 X ，即 $V_h = W_V = Y$ 时，Attention 机制退化为 FM 模块。因此，基于注意力、基于 TokenMixer 和基于 Wukong 的架构可以统一于以下单一理论框架下

$$\text{UniMixing}(X) = \text{reshape}\left(\underbrace{G(X, W_G)}_{\text{全局混合模式}} \underbrace{\begin{bmatrix} \mathbf{x}_1 W_B^1 \\ \vdots \\ \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}} \end{bmatrix}}_{\text{局部混合模式}}, 1, L\right), \quad (17)$$

其中 $G(X, W_G)$ 是异构特征交互投影，用于衡量 token 之间/块之间的交互强度。为了便于分析各种方法之间的差异与联系，我们考虑单头注意力设置。在统一理论框架 (17) 下，它们的差异总结于表 1。对于自注意力、异构注意力和 FM，全局混合模式

表 1: 在统一理论框架下, 基于注意力、基于 TokenMixer 和基于 FM 的方法之间的差异。

异构特征混合	局部混合模式	全局混合模式 $G(X, W_G)$
Self-Attention	XW_V	$\text{softmax}(\frac{(XW_Q)(XW_K)^T}{\sqrt{d}})$
Heterogeneous Attention	$X\tilde{W}_V$	$\text{softmax}(\frac{(X\tilde{W}_Q)(X\tilde{W}_K)^T}{\sqrt{d}})$
TokenMixer	X	G
FM	Y	$XI(XI)^T$

$G(X, W_G)$ 通过计算两个 token 之间的内积相似度得到。TokenMixer 的全局混合模式与输入 token 嵌入无关。

UniMixing-Lite。 如图 3 所示, 可以观察到, 随着块粒度变细, 局部交互参数矩阵 W_B^i 的数量增加, 全局交互参数矩阵 W_G 也随之变大。这会导致局部交互模式冗余。同时, 更大的全局交互矩阵无法高效减少参数量。因此, 在 UniMixing 模块的基础上, 我们设计了轻量级 UniMixing 模块 UniMixing-Lite, 以进一步减少模块参数量和计算成本, 从而提升模型的扩展效率。

为解决局部交互模式的冗余问题, 我们引入基组合模块, 动态生成模块特定的局部混合权重。将 W_B^i 的一组基矩阵定义为 $\{Z_\ell\}_{\ell=1}^b$, 将这些基上的模块特定权重向量定义为 $\{\omega^i\}_{i=1}^{L//B}$, 其中 b 为局部混合权重的基数量, 且 $\omega^i = [\omega_1^i, \dots, \omega_b^i]$ 。此外, 对于全局交互参数 W_G , 我们采用低秩近似以进一步提高效率。随后, UniMixing-Lite 模块可表示为

$$\begin{aligned} \text{UniMixing-Lite}(X) &= \text{reshape}\left(W_r, \text{reshape}\left(\left[\mathbf{x}_1 W_B^{*1} \mid \dots \mid \mathbf{x}_{\frac{L}{B}} W_B^{*\frac{L}{B}}\right], \frac{L}{B}, B\right), 1, L\right), \\ O &= \text{RMSNorm}(X + \text{UniMixing-Lite}(X)), \end{aligned} \quad (18)$$

其中, $W_r = \text{Sinkhorn-Knopp}(A_G B_G)$ 、 $W_B^{*i} = \text{Sinkhorn-Knopp}(\sum_{\ell=1}^b \omega_\ell^i Z_\ell)$ 、 $A_G \in \mathbb{R}^{(L//B) \times r}$ 和 $B_G \in \mathbb{R}^{r \times (L//B)}$ 。 r 为 W_G 的低秩近似的秩。在 UniMixing-Lite 模块中, 我们既保留了 TokenMixer 的低参数量全局交互模式, 也保留了注意力面向异构特征的局部交互能力。该模块可同时发挥基于注意力与基于 TokenMixer 方法的优势。

Pertoken SwiGLU。 在 UniMixing 模块之后, 与 [5] 类似, 引入 pertoken SwiGLU 以建模不同 token 之间的特征异构性。对于每个输入 token $\mathbf{x}_i$, SwiGLU 的形式如下

$$\text{pSwiGLU}(\mathbf{o}_i) = W_{\text{down}}^i ((W_{\text{up}}^i \mathbf{o}_i + b_{\text{up}}^i) \odot \text{Swish}(W_{\text{gate}}^i \mathbf{o}_i + b_{\text{gate}}^i)) + b_{\text{down}}^i, \quad (19)$$

其中, $W_{\text{up}}^i, W_{\text{gate}}^i \in \mathbb{R}^{B \times nB}$ 、 $W_{\text{down}}^i \in \mathbb{R}^{nB \times B}$ 、 $b_{\text{up}}^i, b_{\text{gate}}^i \in \mathbb{R}^{nB}$ 、 $b_{\text{down}}^i \in \mathbb{R}^B$, $\mathbf{o}_i$ 为第 i 个 token 的 UniMixing 输出, n 为超参数。

4.4 SiameseNorm

当前 RankMixer 架构 [4] 缺乏针对深层架构的专门设计，这通常表现为沿模型深度扩展的效果有限。尽管 TokenMixer-Large [5] 尝试通过在 TokenMixer-Large 模块中引入间隔残差和辅助损失来解决该问题，但并未触及问题根源。为在模型深度增加时实现训练稳定性和性能提升，如图 2 所示，我们将 SiameseNorm [26] 引入 UniMixer 架构。如 [26] 所述，SiameseNorm 通过在每一层引入两个耦合流，化解了 Pre-Norm 与 Post-Norm 之间的矛盾。在本小节中，这两个耦合流分别记为 $\bar{X}_i$ 和 $\bar{Y}_i$ ，并由输入嵌入 $\bar{X}_0 = \bar{Y}_0 = X$ 初始化。对于第 ℓ 个模块，SiameseNorm 执行如下更新：

$$\begin{aligned}\tilde{Y}_\ell &= \text{RMSNorm}(\bar{Y}_\ell), \quad O_\ell = \text{UniMixer}(\bar{X}_\ell + \tilde{Y}_\ell) \\ \bar{X}_{\ell+1} &= \text{RMSNorm}(\bar{X}_\ell + O_\ell), \quad \bar{Y}_{\ell+1} = \bar{Y}_\ell + O_\ell.\end{aligned}$$

对于第 M 个 UniMixer 模块，将 $\bar{X}_\ell$ 与 $\bar{Y}_\ell$ 融合以生成最终表示，其形式为

$$X_{\text{output}} = \bar{X}_M + \text{RMSNorm}(\bar{Y}_M). \quad (20)$$

4.5 UniMixer 训练策略

为了使参数矩阵 W_G 和 W_B^i 满足稀疏性要求，我们引入温度系数来控制其稀疏程度。然而，较小的温度会产生更稀疏的权重，同时也会导致梯度变得稀疏、微弱，甚至不稳定。这会增加训练过程的难度，并使优化陷入局部最优。另一方面，我们的实验表明，权重参数的稀疏性对模型性能具有显著的积极影响，如表 3（第 5.3 小节）所示。因此，这种稀疏性不可或缺。一种常用方法是在训练过程中应用线性温度退火：从相对较高的初始温度（例如 $\tau = 1.0$ ）开始，随着训练迭代次数增加，逐步将其线性退火至 0.05，其表达式为

$$\tau_j = \max \left\{ \tau_{\text{start}} - \frac{(\tau_{\text{start}} - \tau_{\text{end}})j}{J}, \tau_{\text{end}} \right\}, \quad (21)$$

其中 τ_j 是第 j 个温度系数， τ_{start} 和 τ_{end} 分别是初始温度和最终温度， J 是温度退火的迭代范围。当数据量不足时，线性退火可能导致前期在高温系数下探索不足，或后期在低温系数下优化欠佳。为了解决这一问题，我们可以先使用较高的温度系数（例如 $\tau = 1.0$ ）冷启动模型训练；当模型充分训练后，再降低温度系数（例如 $\tau = 0.05$ ），并使用高温模型的权重作为初始化来重新训练低温模型。

5 实验

在本节中，我们进行了广泛的实验，将本文的 UniMixer 架构与现有最先进（SOTA）方法进行性能比较，并回答以下问题：

Q1: UniMixer 架构的规模化效率是否优于 SOTA 架构？

Q2: 在不同的全局和局部混合模式设置下，所提方法的性能如何变化？

Q3: 轻量级模块 UniMixing-Lite 是否进一步提高了规模化效率？

Q4: 部署到现实世界的在线系统后, UniMixer/UniMixing-Lite 是否在 A/B 测试中改善了业务指标?

5.1 实验设置

数据集与评估指标。 我们使用快手广告投放场景中真实世界训练数据集的日志数据对用户留存进行建模, 并开展离线和在线评估。该数据集包含一年内收集的超过 7 亿个用户样本, 由数值特征、ID 特征、交叉特征和序列特征等数百种异构特征组成。二元标签 (用户留存 = 1/0) 表示用户在首次激活后的第二天是否返回快手应用。对于推荐模型的规模化评估, 我们采用推荐系统中常用的两项指标, 即 ROC 曲线下面积 (AUC) 和 UAUC (用户级 AUC) 来评估模型性能, 并使用密集参数量、FLOPs 和 MFU 评估模型效率。

基线与实验细节。 我们将本文的 2-block/4-block UniMixer/UniMixing-Lite 架构与以下代表性 SOTA 框架进行比较, 并按建模范式分类

- **基于注意力的架构:** Heterogeneous Attention [1]、HiFormer [1] 和 FAT [2], 它们使用字段特定的查询、键和值投影实现异构特征交互。
- **基于 TokenMixer 的框架:** RankMixer [4] 和 TokenMixer-Large [5], 它们采用基于规则的 token 混合操作进行特征交互。
- **基于 FM 的框架:** Wukong [6], 它将 FMB 的输出与线性投影层的输出拼接, 以扩展交互组件。

所有实验均在由 40 块 GPU 组成的混合分布式训练框架中进行。所有模型采用一致的优化器超参数: 密集部分和稀疏部分均使用 Adam 优化, 学习率设为 0.001。

5.2 性能比较 (对应 Q1)

我们使用约有 1 亿参数的 SOTA 规模化架构与 UniMixer 和 UniMixer-Lite 进行比较, 以探索它们的规模定律。以异构注意力架构作为基础模型。我们的模型与 SOTA 模型的主要性能结果见表 2。可以看出, 在参数预算和计算成本更低的情况下, UniMixer 和 UniMixer-Lite 架构均在多项指标上显著优于其他 SOTA 模型。

随后, 在该广告投放场景中, 除 UniMixer/UniMixer-Lite 外, RankMixer 的性能优于其他所有 SOTA 模型。因此, 我们选择最强的 SOTA 模型与 UniMixer/UniMixer-Lite 进行扩展定律比较。所有模型均在同一数据集上使用一致的超参数进行训练。它们关于参数量和 FLOPs 的扩展曲线如图 4 所示。可以观察到, 随着参数量/FLOPs 增加, 三个模型的 AUC 均呈现出清晰的幂律趋势。UniMixer-Lite 实现了最佳扩展效率, 并表现出更陡峭的提升斜率。根据图 4 所示的参数量与 AUC 之间的关系, RankMixer、UniMixer 和 UniMixer-Lite 中表现良好的 AUC 与参数量/FLOPs 扩展定律可表示如

表 2: 广告投放场景中 $\sim 100\text{M}$ 参数的 **UniMixer** 与 SOTA 模型的性能和效率。

模型	用户留存				效率	
	AUC $\uparrow$	$\Delta\text{AUC}\uparrow$	UAUC $\uparrow$	$\Delta\text{UAUC}\uparrow$	参数量	FLOPs/批次
Heterogeneous Attention [1]	0.744577	–	0.733829	–	132.7M	1.68T
HiFormer [1]	0.741685	-0.2892%	0.731086	-0.2743%	107.5M	1.37T
Wukong [6]	0.744477	-0.0100%	0.733849	0.0020%	107.1M	1.40T
FAT [2]	0.744883	0.0306%	0.734280	0.0451%	138.4M	1.83T
RankMixer [4]	0.749329	0.4752%	0.738938	0.5109%	135.5M	1.68T
TokenMixer-Large [5]	0.748410	0.3833%	0.737940	0.4111%	103.3M	1.27T
UniMixer-2-Blocks 67.5M (本文)	0.749770	0.5193%	0.739331	0.5502%	67.5M	2.07T
UniMixer-2-Blocks 101.5M (本文)	0.750238	0.5661%	0.739983	0.6154%	101.5M	2.50T
UniMixer-Lite-2-Blocks 42.4M (本文)	<u>0.751121</u>	<u>0.6544%</u>	<u>0.740739</u>	<u>0.6910%</u>	42.4M	2.17T
UniMixer-Lite-2-Blocks 76.2M (本文)	<u>0.751401</u>	<u>0.6824%</u>	<u>0.741215</u>	<u>0.7386%</u>	76.2M	2.60T
UniMixer-Lite-4-Blocks 38.2M (本文)	0.752327	0.7750%	0.742091	0.8190%	38.2M	1.26T
UniMixer-Lite-4-Blocks 84.5M (本文)	0.752718	0.8141%	0.742530	0.8701%	84.5M	4.24T

下

$$\begin{aligned}\Delta\text{AUC}_{\text{RankMixer}} &= 0.002718\text{Params}^{0.116043}, \Delta\text{AUC}_{\text{RankMixer}} = 0.002022\text{FLOPs}^{0.116635}, \\ \Delta\text{AUC}_{\text{UniMixer}} &= 0.003032\text{Params}^{0.131973}, \Delta\text{AUC}_{\text{UniMixer}} = 0.002058\text{FLOPs}^{0.125702}, \\ \Delta\text{AUC}_{\text{UniMixer-Lite}} &= 0.003767\text{Params}^{0.141903}, \Delta\text{AUC}_{\text{UniMixer-Lite}} = 0.002338\text{FLOPs}^{0.135327}.\end{aligned}$$

在扩展定律的两个常数中，扩展指数常数对性能增长的影响最为显著，是决定扩展效率的主导因素。UniMixer-Lite 展现出最强的扩展效率，在参数量和 FLOPs 两方面均取得最大的扩展指数与系数。这表明其从模型容量增加中获益最多。

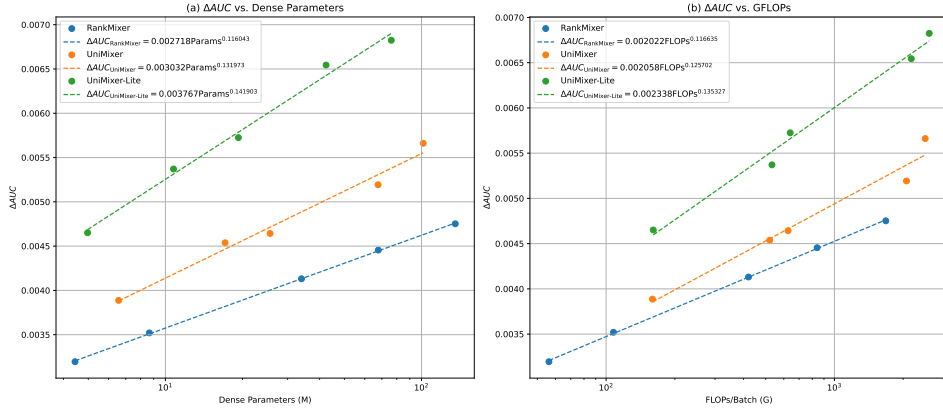


图 4: UniMixer-2-Blocks/UniMixer-Lite-2-Blocks 与 RankMixer 架构中 AUC 与参数量/FLOPs 之间的扩展定律。横轴采用对数尺度。

5.3 消融实验 (对应 Q2)

为探究全局与局部混合权重的性质，以及 UniMixer 中各模块对 AUC 增益的贡献，我们对多个 UniMixer 变体开展消融实验，并测量它们相对于完整 UniMixer 模型的 AUC

变化。所有变体均在相近的设置下训练。结果如表 3 所示，移除任一模块或违反任一参数约束都会导致性能下降，其中较低的温度系数和模型预热对整体性能的影响最为显著。

表 3: UniMixer 6.57M 各组成部分的消融实验。

设置	用户留存				效率	
	AUC↑	Δ AUC↑	UAUC↑	Δ UAUC↑	参数量	FLOPs/批次
UniMixer	0.748464	–	0.738017	–	6.57M	0.16T
不含温度系数	0.746819	-0.1645%	0.736527	-0.1490%	6.57M	0.16T
不含对称性约束	0.747891	-0.0573%	0.737447	-0.0570%	6.57M	0.16T
不含模块特定的局部混合权重	0.748028	-0.0436%	0.737770	-0.0240%	6.53M	0.16T
不含模型预热	0.747608	-0.0856%	0.737180	-0.0837%	6.57M	0.16T
SiameseNorm $\rightarrow$ 后置归一化	0.748191	-0.0273%	0.737660	-0.0357%	6.56M	0.16T

5.4 UniMixing-Lite 模块的性能（针对 Q3）

根据图 4 中的扩展趋势，可以观察到，本文提出的 UniMixing-Lite 架构具有最佳的参数效率与计算效率。在此，我们通过实验研究 $\{Z_\ell\}_{\ell=1}^b$ 的不同基数量 b 、 A_G 和 B_G 的不同秩 r ，以及不同的 UniMixer 模块数量所产生的影响。如表 4 所示，随着基数量 b 和 A_G, B_G 的秩 r 增大，模型性能相应提升。然而，就参数效率而言，增加基数量 b 所带来的 AUC 增益高于增加秩 r 。为观察低秩近似 $A_G B_G$ 以及采用 Sinkhorn–Knopp 操作的基矩阵 $\{Z_\ell\}_{\ell=1}^b$ 对全局和局部混合矩阵重构的影响，我们在一个 2-blocks-UniMixer-Lite 架构中，对第一个 UniMixer 模块重构得到的全局矩阵 $\bar{W}_G$ 和前六个局部混合矩阵 $\bar{W}_B^i$ 进行可视化，其温度系数分别为 $\tau = 1$ 和 $\tau = 0.05$ ，如图 5 所示。输入嵌入维度为 768，块大小为 6；因此，得到 $\bar{W}_G \in \mathbb{R}^{128 \times 128}$ 和 $\bar{W}_B^i \in \mathbb{R}^{6 \times 6}$ ，其中 $A_G \in \mathbb{R}^{128 \times 16}$ 和 $B_G \in \mathbb{R}^{16 \times 128}$ 。根据图 5，尽管模块中采用了低秩近似和基矩阵，Sinkhorn–Knopp 操作仍可确保矩阵接近满秩。此外，对比图 5(a)(b) 与 (c)(d) 可见，温度系数较低的全局和局部混合矩阵比温度系数较高者呈现出更尖锐的交互分布。根据消融实验给出的结果，我们可以得出结论： $\bar{W}_G$ 和 $\bar{W}_B^i$ 的稀疏性能够显著提升模型性能。

另一方面，根据表 4 可以观察到，随着 UniMixer 深度增加，所构建的模型持续呈现清晰的规模扩展趋势，而 RankMixer 则随着 RankMixer 模块堆叠而出现性能下降。采用 2 个和 4 个模块的 UniMixing-Lite 扩展曲线如图 6 所示，这表明沿深度扩展比沿宽度扩展更高效。

5.5 在线 A/B 测试结果（对应 Q4）

为了验证所提 UniMixer 架构的在线性能，我们已在快手的多个广告投放场景中部署 UniMixer 和 UniMixer-Lite。在在线 A/B 测试中，我们在 30 天的观察窗口内使用累计活跃天数（CAD）衡量用户参与度，其中不包括安装日（第 0 天）。在多个场景中，D1–D30 的 CAD 平均提升了超过 15%。

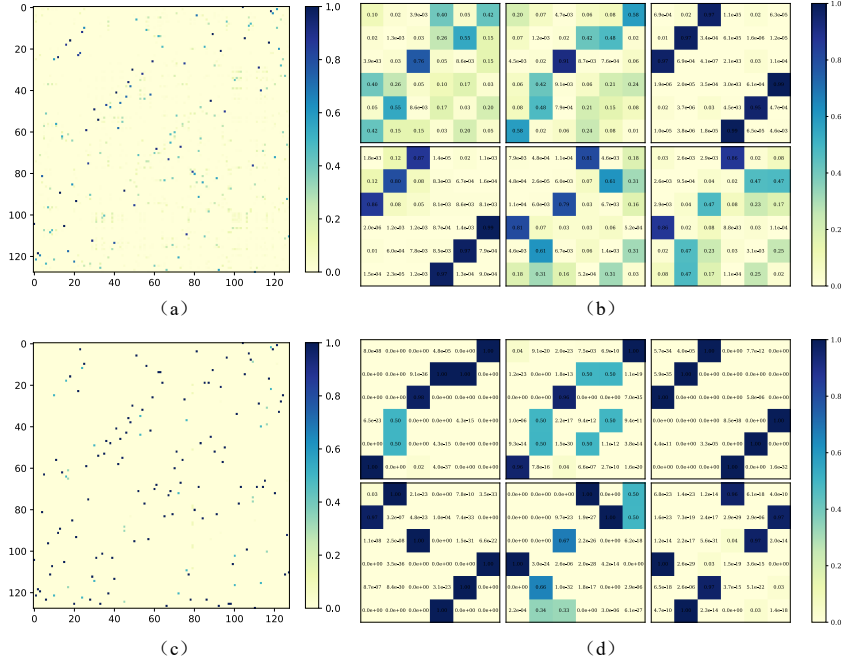


图 5: 不同温度系数下 UniMixer-Lite 的 $\bar{W}_G$ 和 $\bar{W}_B^i$. (a) $\tau = 1$ 条件下的 $\bar{W}_G$; (b) $\tau = 1$ 条件下的 $\{\bar{W}_B^i\}_{i=1}^6$; (c) $\tau = 0.05$ 条件下的 $\bar{W}_G$; (d) $\tau = 0.05$ 条件下的 $\{\bar{W}_B^i\}_{i=1}^6$.

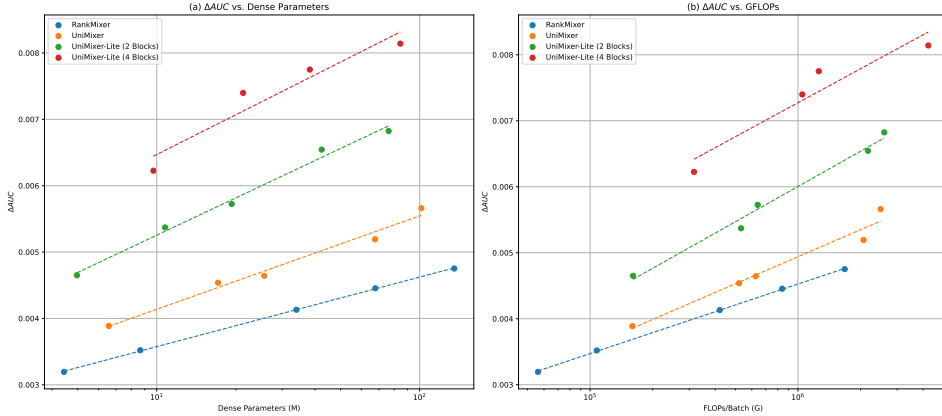


图 6: 采用 2 个和 4 个模块的 RankMixer 与 UniMixer/UniMixer-Lite 中, AUC 与参数量/FLOPs 之间的扩展曲线。

6 结论

本文为推荐系统扩展定律建立了统一扩展框架, 打通了基于注意力、基于 TokenMixer 和基于 FM 的方法之间的联系, 从而能够充分发挥各自的优势。从得到的扩展定律来看, 与 SOTA 架构相比, 本文提出的 UniMixer-Lite 实现了最佳的参数效率与计算效率。我们已在快手的多个场景中部署这些架构, 取得了显著的离线与在线收益。本文不再

表 4: UniMixing-Lite 中基数量、秩和 UniMixer 模块数量的影响。

基数量	用户留存				效率	
	AUC↑	Δ AUC↑	UAUC↑	Δ UAUC↑	参数量	FLOPs/批次
b=2	0.749228	–	0.738776	–	4.968M	0.161T
b=4	0.750230	0.1002%	0.739876	0.1100%	4.973M	0.161T
b=8	0.750283	0.1055%	0.739854	0.1078%	4.98M	0.161T

秩	用户留存				效率	
	AUC↑	Δ AUC↑	UAUC↑	Δ UAUC↑	参数量	FLOPs/批次
r=2	0.748568	–	0.738177	–	4.4525M	0.160T
r=64	0.749002	0.0434%	0.738604	0.0427%	4.705M	0.160T
r=128	0.749228	0.0660%	0.738776	0.0599%	4.9675M	0.161T
r=256	0.749539	0.0971%	0.739437	0.1260%	5.4925M	0.163T

模块数量	用户留存				效率	
	AUC↑	Δ AUC↑	UAUC↑	Δ UAUC↑	参数量	FLOPs/批次
RankMixer-2-Blocks	0.747772	–	0.737322	–	4.4375M	0.056T
RankMixer-4-Blocks	0.746706	-0.1066%	0.736018	-0.1304%	8.6575M	0.108T
UniMixer-Lite-2-Blocks	0.749228	–	0.738776	–	4.9675M	0.161T
UniMixer-Lite-4-Blocks	0.750803	0.1575%	0.740594	0.1818%	9.715M	0.316T
UniMixer-Lite-8-Blocks	0.750875	0.1647%	0.740602	0.1826%	19.2125M	0.629T

孤立地看待推荐模型中现有的扩展模块（例如 Heterogeneous Attention、TokenMixer、Wukong），而是建立了一个统一理论框架，为推荐系统的扩展设计提供指导。我们相信，统一架构能够帮助推荐系统社区迎来属于自己的“attention moment”。统一模块 UniMixer 可作为专为推荐领域定制的基础模块，其适用范围还可进一步扩展至用户行为序列建模与生成式推荐任务。

References

- [1] Huan Gui, Ruoxi Wang, Ke Yin, Long Jin, Maciej Kula, Taibai Xu, Lichan Hong, and Ed H Chi. Hiformer: Heterogeneous feature interactions learning with transformers for recommender systems. *arXiv preprint arXiv:2311.05884*, 2023.
- [2] Bencheng Yan, Yuejie Lei, Zhiyuan Zeng, Di Wang, Kaiyi Lin, Pengjie Wang, Jian Xu, and Bo Zheng. From scaling to structured expressivity: Rethinking transformers for ctr prediction. *arXiv preprint arXiv:2511.12081*, 2025.
- [3] Liren Yu, Wenming Zhang, Silu Zhou, Tao Zhang, Zhixuan Zhang, and Dan Ou. Hhft: Hierarchical heterogeneous feature transformer for recommendation systems. *arXiv preprint arXiv:2511.20235*, 2025.

- [4] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, et al. Rankmixer: Scaling up ranking models in industrial recommenders. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 6309–6316, 2025.
- [5] Yuchen Jiang, Jie Zhu, Xintian Han, Hui Lu, Kunmin Bai, Mingyu Yang, Shikang Wu, Ruihao Zhang, Wenlin Zhao, Shipeng Bai, et al. Tokenmixer-large: Scaling up large ranking models in industrial recommenders. *arXiv preprint arXiv:2602.06563*, 2026.
- [6] Buyun Zhang, Liang Luo, Yuxin Chen, Jade Nie, Xi Liu, Daifeng Guo, Yanli Zhao, Shen Li, Yuchen Hao, Yantao Yao, et al. Wukong: Towards a scaling law for large-scale recommendation. *Proceedings of the 41st International Conference on Machine Learning*, 235:1–20, 2024.
- [7] Bojian Hou, Xiaolong Liu, Xiaoyi Liu, Jiaqi Xu, Yasmine Badr, Mengyue Hang, Sudhanshu Chanpuriya, Junqing Zhou, Yuhang Yang, Han Xu, et al. Kunlun: Establishing scaling laws for massive-scale recommendation systems through unified architecture design. *arXiv preprint arXiv:2602.10016*, 2026.
- [8] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. Actions speak louder than words: trillion-parameter sequential transducers for generative recommendations. In *Proceedings of the 41st International Conference on Machine Learning*, ICML’24, 2024.
- [9] Qin Ding, Kevin Course, Linjian Ma, Jianhui Sun, Rouchen Liu, Zhao Zhu, Chunxing Yin, Wei Li, Dai Li, Yu Shi, et al. Bending the scaling law curve in large-scale recommendation systems. *arXiv preprint arXiv:2602.16986*, 2026.
- [10] Xiao Lv, Jiangxia Cao, Shijie Guan, Xiaoyou Zhou, Zhiguang Qi, Yaqiang Zang, Ben Wang, and Guorui Zhou. Marm: Unlocking the recommendation cache scaling-law through memory augmentation and scalable complexity. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, CIKM ’25, page 2022–2031, New York, NY, USA, 2025. Association for Computing Machinery.
- [11] Zhaoqi Zhang, Haolei Pei, Jun Guo, Tianyu Wang, Yufei Feng, Hui Sun, Shaowei Liu, and Aixin Sun. Onetrans: Unified feature interaction and sequence modeling with one transformer in industrial recommender. *arXiv preprint arXiv:2510.26104*, 2025.

- [12] Songpei Xu, Shijia Wang, Da Guo, Xianwen Guo, Qiang Xiao, Bin Huang, Guanlin Wu, and Chuanjiang Luo. Climber: Toward efficient scaling laws for large recommendation models. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, CIKM '25, page 6193–6200. Association for Computing Machinery, 2025.
- [13] Yunwen Huang, Shiyong Hong, Xijun Xiao, Jinqiu Jin, Xuanyuan Luo, Zhe Wang, Zheng Chai, Shikang Wu, Yuchao Zheng, and Jingjian Lin. Hyformer: Revisiting the roles of sequence modeling and feature interaction in ctr prediction. *arXiv preprint arXiv:2601.12681*, 2026.
- [14] Lee Xiong, Zhirong Chen, Rahul Mayuranath, Shangran Qiu, Arda Ozdemir, Lu Li, Yang Hu, Dave Li, Jingtao Ren, Howard Cheng, et al. Llatte: Scaling laws for multi-stage sequence modeling in large-scale ads recommendation. *arXiv preprint arXiv:2601.20083*, 2026.
- [15] Ilya O Tolstikhin, Neil Houlsby, Alexander Kolesnikov, Lucas Beyer, Xiaohua Zhai, Thomas Unterthiner, Jessica Yung, Andreas Steiner, Daniel Keysers, Jakob Uszkoreit, et al. Mlp-mixer: An all-mlp architecture for vision. *Advances in neural information processing systems*, 34:24261–24272, 2021.
- [16] Xintian Han, Honggang Chen, Quan Lin, Jingyue Gao, Xiangyuan Ren, Lifei Zhu, Zhisheng Ye, Shikang Wu, XiongHao Xie, Xiaochu Gan, et al. Lemur: Large scale end-to-end multimodal recommendation. *arXiv preprint arXiv:2511.10962*, 2025.
- [17] Steffen Rendle. Factorization machines. In *2010 IEEE International Conference on Data Mining*, pages 995–1000. IEEE, 2010.
- [18] Yuchin Juan, Yong Zhuang, Wei-Sheng Chin, and Chih-Jen Lin. Field-aware factorization machines for ctr prediction. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 43–50, 2016.
- [19] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. Deepfm: a factorization-machine based neural network for ctr prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, pages 1725–1731, 2017.
- [20] Weiping Song, Chence Shi, Zhiping Xiao, Zhijian Duan, Yewen Xu, Ming Zhang, and Jian Tang. Autoint: Automatic feature interaction learning via self-attentive neural networks. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, pages 1161–1170, 2019.
- [21] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. Deep & cross network for ad click predictions. In *Proceedings of the ADKDD'17*, pages 1–7. 2017.

- [22] Ruoxi Wang, Rakesh Shivanna, Derek Cheng, Sagar Jain, Dong Lin, Ed H Chi, and Min Chi. Dcn v2: Improved deep & cross network and practical lessons for web-scale ctr prediction. In *Proceedings of the Web Conference 2021*, pages 1785–1797, 2021.
- [23] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [24] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Oriol Vinyals, Jack W. Rae, and Laurent Sifre. Training compute-optimal large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, Red Hook, NY, USA, 2022.
- [25] Xianyang Qi, Yuan Tian, Zhaoyu Hu, Zhirui Kuai, Chang Liu, Hongxiang Lin, and Lei Wang. Mtmixatt: Integrating mixture-of-experts with multi-mix attention for large-scale recommendation, 2025.
- [26] Tianyu Li, Dongchen Han, Zixuan Cao, Haofeng Huang, Mengyu Zhou, Ming Chen, Erchao Zhao, Xiaoxi Jiang, Guanjun Jiang, and Gao Huang. Siamesenorm: Breaking the barrier to reconciling pre/post-norm, 2026.

A TokenMixer 等价变换的数值示例

给定如下输入隐藏状态 $X \in \mathbb{R}^{2 \times 6}$

$$X = \left[\begin{array}{ccc|ccc} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 & x_{10} & x_{11} & x_{12} \end{array} \right],$$

其中 x_i 为标量。随后，经过 TokenMixer 操作的输入隐藏状态 X 被转换为

$$\text{TokenMixer}(X) = \left[\begin{array}{ccc|ccc} x_1 & x_2 & x_3 & x_7 & x_8 & x_9 \\ x_4 & x_5 & x_6 & x_{10} & x_{11} & x_{12} \end{array} \right].$$

TokenMixer 的输出可以展平为向量

$$\text{flatten}(\text{TokenMixer}(X)) = [x_1, x_2, x_3, x_7, x_8, x_9, x_4, x_5, x_6, x_{10}, x_{11}, x_{12}]^T \quad (22)$$

另一方面，通过乘以一个 12×12 矩阵，可以将向量 $\text{flatten}(X)$ 转换为 $\text{flatten}(\text{TokenMixer}(X))$ ，其形式为

$$\underbrace{\left[\begin{array}{ccc|ccc|ccc|ccc} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ \hline 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{array} \right]}_{W^{\text{perm}}} \underbrace{\left[\begin{array}{c} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \\ x_{10} \\ x_{11} \\ x_{12} \end{array} \right]}_{\text{flatten}(X)} = \underbrace{\left[\begin{array}{c} x_1 \\ x_2 \\ x_3 \\ x_7 \\ x_8 \\ x_9 \\ x_4 \\ x_5 \\ x_6 \\ x_{10} \\ x_{11} \\ x_{12} \end{array} \right]}_{\text{flatten}(\text{TokenMixer}(X))} \quad (23)$$

根据 (22) 和 (23)，该数值示例中的 TokenMixer 操作可等价转换为矩阵乘法形式。此外，置换矩阵 $W^{\text{perm}} \in \mathbb{R}^{12 \times 12}$ 可等价分解为以下两个小矩阵的 Kronecker 积

$$W^{\text{perm}} = \underbrace{\left[\begin{array}{cccc} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{array} \right]}_{\text{全局混合矩阵}} \otimes \underbrace{\left[\begin{array}{ccc} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{array} \right]}_{\text{局部混合矩阵}}.$$

B UniMixing 模块的计算流水线优化

如下定义 $W_G \in \mathbb{R}^{(L//B) \times L//B}$ 和 $W_B^i \in \mathbb{R}$

$$W_G = \begin{bmatrix} w_{(1,1)}^G & \cdots & w_{(1,L//B)}^G \\ \cdots & \cdots & \cdots \\ w_{(L//B,1)}^G & \cdots & w_{(L//B,L//B)}^G \end{bmatrix}, W_B^i = \begin{bmatrix} v_{(1,1)}^i & \cdots & v_{(1,B)}^i \\ \cdots & \cdots & \cdots \\ v_{(B,1)}^i & \cdots & v_{(B,B)}^i \end{bmatrix}, \quad (24)$$

其中 w_{ij} 和 v_{ij} 为标量。根据 (12)，将 $\text{flatten}(X)$ 均匀分割为 $L//B$ 个向量，可重写为

$$\text{flatten}(X) = \left[\mathbf{x}_1 \mid \mathbf{x}_2 \mid \cdots \mid \mathbf{x}_{\frac{L}{B}} \right]^\top, \quad (25)$$

其中 $\mathbf{x}_i$ 是维度为 B 的行向量。

根据 UniMing 的原始表达式 (11)，项 $(W_G \otimes \{W_B^i\}_{i=1}^{L//B}) \text{flatten}(X)$ 可重写为

$$\begin{aligned} (W_G \otimes \{W_B^i\}_{i=1}^{L//B}) \text{flatten}(X) &= \begin{bmatrix} w_{(1,1)}^G W_B^1 & \cdots & w_{(1,L//B)}^G W_B^{L//B} \\ \cdots & \cdots & \cdots \\ w_{(L//B,1)}^G W_B^1 & \cdots & w_{(L//B,L//B)}^G W_B^{L//B} \end{bmatrix} \begin{bmatrix} \mathbf{x}_1^\top \\ \vdots \\ \mathbf{x}_{\frac{L}{B}}^\top \end{bmatrix} \\ &= \begin{bmatrix} w_{(1,1)}^G W_B^1 \mathbf{x}_1^\top + \cdots + w_{(1,L//B)}^G W_B^{L//B} \mathbf{x}_{\frac{L}{B}}^\top \\ \cdots \\ w_{(L//B,1)}^G W_B^1 \mathbf{x}_1^\top + \cdots + w_{(L//B,L//B)}^G W_B^{L//B} \mathbf{x}_{\frac{L}{B}}^\top \end{bmatrix} \in \mathbb{R}^{L \times 1} \end{aligned} \quad (26)$$

另一方面，可以得到如下表达式

$$\begin{aligned} W_G \text{reshape} \left(\left[\mathbf{x}_1 W_B^{1\top} \mid \cdots \mid \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}\top} \right], \frac{L}{B}, B \right) &= \begin{bmatrix} w_{(1,1)}^G & \cdots & w_{(1,L//B)}^G \\ \cdots & \cdots & \cdots \\ w_{(L//B,1)}^G & \cdots & w_{(L//B,L//B)}^G \end{bmatrix} \begin{bmatrix} \mathbf{x}_1 W_B^{1\top} \\ \vdots \\ \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}\top} \end{bmatrix} \\ &= \begin{bmatrix} w_{(1,1)}^G \mathbf{x}_1 W_B^{1\top} + \cdots + w_{(1,L//B)}^G \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}\top} \\ \cdots \\ w_{(L//B,1)}^G \mathbf{x}_1 W_B^{1\top} + \cdots + w_{(L//B,L//B)}^G \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}\top} \end{bmatrix} \in \mathbb{R}^{\frac{L}{B} \times B} \end{aligned} \quad (27)$$

((26)) 中的元素与 (27) 中的元素满足

$$w_{(i,1)}^G W_B^1 \mathbf{x}_1^\top + \cdots + w_{(i,L//B)}^G W_B^{L//B} \mathbf{x}_{\frac{L}{B}}^\top = (w_{(i,1)}^G \mathbf{x}_1 W_B^{1\top} + \cdots + w_{(i,L//B)}^G \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}\top})^\top, \quad (28)$$

由此得到

$$(W_G \otimes \{W_B^i\}_{i=1}^{L//B}) \text{flatten}(X) = \text{reshape} \left(W_G \text{reshape} \left(\left[\mathbf{x}_1 W_B^{1\top} \mid \cdots \mid \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}\top} \right], \frac{L}{B}, B \right), L, 1 \right)$$

由于 W_B^i 和 $W_B^{i\top}$ 均为可学习参数，参数转置不会影响模型。因此，经过计算流水线优化后的 UniMixing 模块可表示为

$$\text{UniMixing}(X) = \text{reshape} \left(W_G \text{reshape} \left(\left[\mathbf{x}_1 W_B^1 \mid \mathbf{x}_2 W_B^2 \mid \cdots \mid \mathbf{x}_{\frac{L}{B}} W_B^{\frac{L}{B}} \right], \frac{L}{B}, B \right), 1, L \right). \quad (29)$$